

EdgeSim-DCCC: A Python Simulation Framework for Deadline-Aware Cooperative Edge Caching Research

Simulator Documentation and Technical Reference

EdgeSim-DCCC Project

May 2026 — Version 1.0

Abstract

We present **EdgeSim-DCCC**, an open-source Python simulation framework designed for reproducible, rigorous research on deadline-aware cooperative edge caching in multi-region mobile edge computing networks. The framework provides a deterministic world model (`build_world`), a pluggable policy interface, three built-in cache replacement algorithms (LRU, LFU, and ICE-style popularity-weighted eviction), and a Deep Reinforcement Learning (DRL) integration layer based on Deep Q-Networks (DQN). The simulator supports seven caching policies ranging from classic baselines (LRU, LFU) to cooperative heuristics (DCCC_HASH, DCCC_NEAREST) and a family of deadline-aware policies (DEADLINE_HEURISTIC, DEADLINE_NO_OVERLAP, DEADLINE_DCCC). All policies share an identical world instance per random seed, so cross-policy performance variance reflects routing and caching decisions rather than workload sampling noise. An episodic multi-seed evaluation harness produces paired t-test and Wilcoxon signed-rank significance tables automatically. This paper documents the architecture, data models, simulation engine, policy framework, DRL training loop, evaluation harness, and all 59 configurable parameters.

Contents

1	Introduction	4
2	Architecture Overview	4
2.1	Data-Flow Summary	4
3	World Model	5
3.1	Content Generation	5
3.2	Edge Node Topology	5
3.2.1	Initial Cache Distribution	6
3.3	Request Generation	6
4	Simulation Engine	6

4.1	Entry Points	6
4.2	Latency Model	6
4.3	Request Processing Loop	7
4.4	Load Model	7
4.5	Energy Model	7
4.6	Metrics Collected	7
5	Policy Framework	8
5.1	Policy Plugin Interface	8
5.2	Cache Replacement Algorithms	8
5.2.1	Least Recently Used (LRU)	8
5.2.2	Least Frequently Used (LFU)	8
5.2.3	ICE-Style Popularity-Weighted Eviction (<code>add_ice_style</code>)	8
5.3	Baseline Policies	8
5.3.1	LRU	8
5.3.2	LFU	9
5.3.3	DCCC_HASH	9
5.3.4	DCCC_NEAREST	9
5.4	Deadline-Aware Policies	9
5.4.1	DEADLINE_HEURISTIC	9
5.4.2	DEADLINE_NO_OVERLAP	9
5.4.3	DEADLINE_DCCC	9
5.5	Deadline-Aware Scoring Function	9
5.6	Newton-Cooling Popularity	10
5.7	Common-Overlap Ratio	10
6	Deep Reinforcement Learning Integration	10
6.1	Problem Formulation as POMDP	10
6.2	7-Dimensional State Vector	10
6.3	3-Action Space	11
6.4	Network Architecture	11
6.5	Experience Replay	11
6.6	Reward Function	12
6.7	Training Loop	12
7	Evaluation Harness	12
7.1	Multi-Seed Runner	12
7.2	Output Files	12
7.3	Statistical Tests	13
7.3.1	95% Confidence Intervals	13
7.3.2	Paired t-Test	13
7.3.3	Wilcoxon Signed-Rank Test	13
7.4	Baseline Selection	13
8	Configuration Reference	14
8.1	Network and Topology	14
8.2	Traffic Model	15
8.3	Latency and Deadline	15
8.4	Cache Management	16

8.5	Deadline-Aware Score Weights	16
8.6	Energy Model	16
8.7	DRL Hyperparameters	16
8.8	Reward Shaping	17
8.9	Experiment Harness	17
9	Reproducibility and Usage	17
9.1	Installation	17
9.2	CLI Reference	17
9.3	Example Runs	17
9.3.1	Full Research Run	17
9.3.2	Quick Smoke Test (No DRL)	18
9.3.3	Deadline-Aware Ablation	18
9.3.4	Single-Policy Evaluation with Custom Config	18
9.3.5	Running the Experiment Harness Directly	18
9.4	Output Directory Structure	18
9.5	Extending the Framework	19
9.5.1	Adding a New Policy	19
9.5.2	Custom DRL Hook	19
9.5.3	Programmatic API	19
9.6	Reproducibility Checklist	20
10	Conclusion	20
A	Module Index	21
B	Policy Summary Table	21

1 Introduction

Mobile edge computing (MEC) places computation and storage resources at the network periphery, enabling content delivery with sub-millisecond latencies compared to centralised cloud infrastructure. Cooperative edge caching extends this paradigm: edge nodes share their cache state to serve requests that miss locally, provided the additional cooperative-lookup latency still meets the application deadline. Designing and evaluating such policies requires a simulation environment that faithfully models heterogeneous node capacities, region-skewed request distributions, content popularity dynamics, deadline constraints at two tiers, and the interactions between routing decisions and cache state.

Existing simulators either abstract away deadline constraints, treat each node independently, or tightly couple the workload model to a specific policy, making ablation studies difficult. EdgeSim-DCCC addresses these gaps with four design principles:

1. **World isolation.** A single call to `build_world(seed, config)` produces a self-contained, reproducible world: content catalogue, edge node topology with pre-seeded caches, and a full request stream. Any number of policies can be evaluated against the same world, ensuring that performance differences are attributable to the policy alone.
2. **Decoupled DRL.** The DRL agent is injected into the simulation loop as a callback (`drl_hook`), keeping the core simulator free of PyTorch dependencies for baseline runs.
3. **Pluggable policies.** Adding a new policy requires implementing a node-selection rule and a cache-insertion rule; the request loop, latency model, and metric collection are inherited unchanged.
4. **Statistical rigour.** The evaluation harness runs every (policy, seed) pair through the same framework and automatically computes paired significance tests.

The remainder of this paper is organised as follows. Section 2 gives an architecture overview. Sections 3–4 describe the world model and simulation engine. Section 5 covers the policy framework. Section 6 details the DRL integration. Section 7 describes the evaluation harness. Section 8 provides the full configuration reference. Section 9 gives reproducibility and usage instructions. Section 10 concludes.

2 Architecture Overview

EdgeSim-DCCC is organised into six top-level packages. Figure 1 summarises the data flow between components.

2.1 Data-Flow Summary

1. `main.py` (or `runner.py`) parses the JSON config and the CLI flags.
2. For each (policy, seed) pair: `build_world` creates contents, nodes, and a request stream using a seeded RNG.
3. For DEADLINE_DCCC, `train_drl_agents` trains one DQN per node using abbreviated episodes, then `make_eval_hook` wraps the greedy agents into a callback.
4. `run_simulation` processes the request stream. For each request it calls `_select_node`, applies the routing decision tree, and calls `_insert_after_serve`.

Module dependency and data-flow diagram

Package	Key module	Responsibility
models/	content.py, edge_node.py, request.py	Data classes
sim/	simulator.py	World generation, simulation loop
policies/	deadline_dccc_policy.py	Scoring primitives
drl/	dqn_agent.py, trainer.py, replay_buffer.py	DQN training
experiments/	runner.py, stats.py	Multi-seed harness
configs/	simulation_config.json	59-key parameter store

Figure 1: Module responsibilities and dependency graph (consumer $\rightarrow$ provider).

5. `SimMetrics.to_row()` converts accumulated counters and latency lists into a flat metric dictionary.
6. `run_multi_seed` aggregates rows into DataFrames and writes per-seed, summary, and significance CSVs.

3 World Model

The world model is responsible for generating a deterministic, statistically realistic simulation environment. All three sub-components (content catalogue, node topology, request stream) are driven by a single seeded Python `random.Random` instance so that the entire world can be reproduced exactly from a seed integer.

3.1 Content Generation

The content catalogue contains $N_c = 120$ items by default. Each content item c_i is characterised by:

- **Size** $s_i \in [50, 2500]$ MB, drawn uniformly at random.
- **Base popularity** following a Zipf distribution with exponent $\alpha = 0.85$:

$$p_i = \frac{1}{(i+1)^\alpha}, \quad i = 0, 1, \dots, N_c - 1. \quad (1)$$

- **Region bias vector** $\mathbf{b}_i \in [0.1, 1.0]^{N_r}$, one scalar per region, drawn independently and uniformly; controls how much more likely a user in that region is to request content i .
- **Content type** $\in \{\text{video}, \text{image}, \text{iot}, \text{audio}\}$, sampled uniformly.

The Zipf exponent $\alpha = 0.85$ produces a moderately heavy-tailed distribution that is less extreme than $\alpha = 1.0$ (standard Zipf), reflecting empirically observed Internet video-on-demand catalogues.

3.2 Edge Node Topology

Five edge nodes are created with heterogeneous capacities: nodes 0, 1, 3 hold 9000 MB each; nodes 2 and 4 hold 2700 MB each. Each node is assigned a region $r_j = j \bmod N_r$, spreading the five nodes across the three regions (0, 1, 2).

3.2.1 Initial Cache Distribution

Before any policy begins processing requests, every content item is placed into the cluster using the DCCC-hash rule: content i is stored at node $i \bmod N_n$ (primary). If $p_i > \theta_{sec} = 0.25$ (the secondary-cache threshold), the item is also stored at node $(i + 1) \bmod N_n$ (secondary). This seed distribution is policy-agnostic, ensuring all policies start from an identical cache state.

3.3 Request Generation

The request stream of $R = 6000$ requests is generated as follows:

1. **User region selection:** region r_u is sampled according to weights $[0.60, 0.25, 0.15]$ (region 0 is dominant, reflecting a hot-spot scenario).
2. **Content selection:** for user region r_u , each content item c_i receives an adjusted weight

$$w_i = p_i \cdot \left(1 + \mu_r \cdot b_i^{(r_u)}\right), \quad (2)$$

where $\mu_r = 2.5$ is the region-weight multiplier. Content c_i is then selected by weighted sampling.

3. Each request is assigned the strict deadline $D_1 = 35$ ms.

4 Simulation Engine

4.1 Entry Points

build_world(seed, config) Constructs and returns a `World` dataclass containing the content list, node list, request stream, and a content-id look-up dictionary. Calling this function twice with the same seed yields byte-identical worlds.

run_simulation(policy, seed, config, world, drl_hook, log_callback) Runs one (policy, seed) replication and returns a `SimMetrics` object. If `world` is supplied it is reused; otherwise it is built internally.

edge_latency_ms(node, content, user_region, config) Computes the latency of serving content from `node` to `user_region` (see Section 4.2).

4.2 Latency Model

The one-way edge-serve latency for node j , content i , user region r_u is:

$$L_{j,i,r_u} = L_{\text{base}} + \tau_{\text{edge}} \cdot s_i + \delta_{\text{load}} \cdot \ell_j + \delta_{\text{region}} \cdot |r_j - r_u| \quad (3)$$

where:

- $L_{\text{base}} = 8$ ms is the base edge processing latency,
- $\tau_{\text{edge}} = 0.012$ ms/MB is the edge transfer coefficient,
- $\delta_{\text{load}} = 10$ ms is the per-unit load delay multiplier,
- $\ell_j \in [0, 1.5]$ is the current normalised load of node j ,
- $\delta_{\text{region}} = 4$ ms/hop is the inter-region delay.

For cooperative retrieval, the cooperative latency is:

$$L_{\text{coop}} = D_{\text{coop_path}} + \frac{L_{j,i,r_u}}{2} \quad (4)$$

where $D_{\text{coop_path}} = 35$ ms is the inter-node messaging overhead and the halving approximates a round-trip offset. Cloud fallback costs:

$$L_{\text{cloud}} = L_{\text{cloud_base}} + \tau_{\text{cloud}} \cdot s_i = 120 + 0.025 \cdot s_i \text{ ms.} \quad (5)$$

4.3 Request Processing Loop

Algorithm 1 describes the main simulation loop. For each request the simulator (i) decays node loads, (ii) builds a candidate node set, (iii) selects the target node via the policy hook, (iv) resolves a routing path (edge / cooperative / cloud), (v) updates the cache, and (vi) records metrics.

4.4 Load Model

Node load ℓ_j is a normalised scalar in $[0, \ell_{\text{max}}]$ where $\ell_{\text{max}} = 1.5$. After each time step, all loads decay by a factor $\lambda_{\text{decay}} = 0.90$ plus a small uniform noise term with amplitude 5×10^{-3} to prevent bit-identical trajectories when two policies make identical routing choices. Each served request (edge or cooperative) increments the serving node's load by $\Delta_\ell = 0.25$.

4.5 Energy Model

Energy cost per request is modelled as:

$$e_{\text{req}} = \alpha_{\text{tier}} \cdot s_i \quad (6)$$

where $\alpha_{\text{local}} = 0.02$ for edge and cooperative serves and $\alpha_{\text{cloud}} = 0.09$ for cloud fetches, reflecting the well-established observation that offloading to the cloud incurs 4.5× more energy per megabyte transferred.

4.6 Metrics Collected

The `SimMetrics.to_row()` method produces the following columns per (policy, seed) replication:

Metric	Formula	Better
hit_rate	$(H_e + H_c)/R$	↑
edge_hit_rate	H_e/R	↑
coop_hit_rate	H_c/R	↑
cloud_fetch_rate	H_{cl}/R	↓
deadline_satisfaction	D_{met}/R	↑
avg_latency_ms	$\bar{L}$	↓
p95_latency_ms	$L_{0.95}$	↓
p99_latency_ms	$L_{0.99}$	↓
avg_energy	$\bar{e}$	↓
cache_evictions	$\sum \text{evictions}$	↓
load_imbalance	$\max(\text{serves}_j)/\bar{\text{serves}}$	↓

5 Policy Framework

5.1 Policy Plugin Interface

All policies share the same simulation loop. A policy is distinguished by two functions:

- `_select_node`** Given the candidate node set, all nodes, the content, user region, popularity, and D_1 , returns the $(node, score)$ pair that the policy routes to.
- `_insert_after_serve`** Given the serving node, content, popularity, time step, and hit type, performs the cache insertion (possibly evicting items) and returns the number of evictions.

To add a new policy, a developer adds a branch in each of these two functions and registers the policy name in `configs/simulation_config.json` under the `policies` list.

5.2 Cache Replacement Algorithms

5.2.1 *Least Recently Used (LRU)*

LRU (`add_lru`) maintains a `last_access` dictionary keyed by content ID. On insertion, if capacity is exhausted, the item with the smallest `last_access` timestamp is evicted. On a cache hit, the access time is refreshed. This is the classic $O(1)$ approximation to Belady’s optimal policy under recency-dominant workloads.

5.2.2 *Least Frequently Used (LFU)*

LFU (`add_lfu`) maintains a `frequency` counter per item. The item with the lowest cumulative access count is evicted when the cache is full. LFU is well-suited to steady-state Zipf workloads but can retain stale items that were popular early in the trace.

5.2.3 *ICE-Style Popularity-Weighted Eviction (add_ice_style)*

The ICE-style eviction (`add_ice_style`) assigns each cached item a reward score:

$$\rho_i(t) = \text{pop}(i, t) \cdot \log_e(1 + s_i) \quad (7)$$

The term $\log_e(1 + s_i)$ implements a size-aware priority: a large, popular item is more valuable to retain than a small item of equal popularity, because it saves more bandwidth if served from the edge. The item with the lowest ρ is evicted. The score is refreshed on every cache hit so that content which has cooled in popularity (via Newton cooling) eventually becomes eviction-eligible.

An ICE-style cache gate $\theta_{\text{ice}} = 0.6$ is applied: an edge hit does not trigger re-insertion unless $\text{pop} \cdot \log_e(1 + s_i) > \theta_{\text{ice}}$. This prevents churning items that are already hot in the local cache.

5.3 Baseline Policies

5.3.1 *LRU*

LRU routes to the nearest accessible node (minimum latency) within the candidate set and uses `add_lru` for cache management. It does not use cooperative lookup: if the selected node misses, the request goes directly to cloud.

5.3.2 LFU

Identical routing to LRU but uses `add_lfu` for eviction.

5.3.3 DCCC_HASH

Hash-based cooperative caching: node $j^* = c_i \bmod N_n$ is always selected, regardless of load, latency, or current cache state. If j^* misses, a cooperative lookup scans all nodes and routes to the best holder within $D_2 = 95$ ms; otherwise falls back to cloud. Cache management uses ICE-style insertion.

5.3.4 DCCC_NEAREST

Latency-aware cooperative caching: the node with minimum L_{j,i,r_u} among the candidate set is selected. Cooperative lookup and ICE-style insertion are used as in DCCC_HASH.

5.4 Deadline-Aware Policies

5.4.1 DEADLINE_HEURISTIC

Selects the node maximising the deadline-aware score $S(j, i, r_u, t)$ (Section 5.5) including the overlap penalty term. Cache management uses ICE-style insertion. No DRL component.

5.4.2 DEADLINE_NO_OVERLAP

Identical to DEADLINE_HEURISTIC except the w_{overlap} term is dropped (flag `drop_overlap` set to `True`). This ablation tests whether the soft cache-partitioning incentive in the overlap term is beneficial.

5.4.3 DEADLINE_DCCC

The proposed policy. Selects the node via the deadline-aware score (same as DEADLINE_HEURISTIC), but the DRL hook may override the routing action for the selected node before the request is served. Section 6 details the DRL component.

5.5 Deadline-Aware Scoring Function

The heuristic node-selection score is:

$$S(j) = w_p \cdot \text{pop} + w_h \cdot h_j - w_o \cdot \text{overlap}_j - w_l \cdot L_j - w_{\ell^2} \cdot \ell_j^2 - w_r \cdot \max(0, L_j - D_1) \quad (8)$$

where:

- $h_j = 1$ if node j caches content i , else $\min(0.5, \text{pop})$ (partial credit for a likely-warm miss).
- overlap_j is the fraction of node j 's cache also held by at least one other candidate node (Section 5.7).
- The ℓ_j^2 term penalises load *non-linearly*, imposing a heavy cost only near saturation while tolerating moderate utilisation.

- $\max(0, L_j - D_1)$ is the deadline-risk penalty: zero when the node can serve within D_1 , positive and growing when it cannot.

Default weights: $w_p = 2.0$, $w_h = 6.0$, $w_o = 0.5$, $w_l = 0.08$, $w_{\ell^2} = 1.2$, $w_r = 0.4$. The $w_h = 6.0$ dominates, strongly preferring nodes that already cache the content to avoid cold-miss cooperative overhead.

5.6 Newton-Cooling Popularity

The instantaneous popularity of content i at time step t is:

$$\text{pop}(i, t) = \left(p_i + \frac{n_i(t)}{\sigma} \right) \cdot e^{-\lambda t} \quad (9)$$

where $n_i(t)$ is the cumulative request count for content i up to time t , $\sigma = 20$ is the request-scale normaliser, and $\lambda = 4 \times 10^{-4}$ is the cooling rate. The formula combines the static Zipf rank with a dynamic demand signal: a content that receives many requests quickly gains a popularity boost. The exponential decay ensures that historically popular but currently cold content eventually falls below the ICE eviction threshold.

5.7 Common-Overlap Ratio

$$\text{overlap}(j, \mathcal{N}_c) = \frac{|\text{cache}(j) \cap \bigcup_{k \neq j, k \in \mathcal{N}_c} \text{cache}(k)|}{|\text{cache}(j)|} \quad (10)$$

A value of 1.0 means every item node j holds is also available at some other candidate node. The overlap penalty in $S(j)$ softly discourages routing to nodes that offer no unique cached content to the cooperative cluster, nudging the cluster towards a natural soft-partitioning of the content catalogue across nodes.

6 Deep Reinforcement Learning Integration

6.1 Problem Formulation as POMDP

The edge caching routing problem is formulated as a Partially Observable Markov Decision Process (POMDP). Each edge node operates an independent DQN agent that observes its own local state. The partial observability arises because no agent has access to the global cache state or the request stream of other nodes.

State space $\mathcal{S}$ 7-dimensional continuous vector (Section 6.2).

Action space $\mathcal{A}$ $\{0, 1, 2\}$ (Section 6.3).

Reward r Shaped scalar (Section 6.6).

Transition Implicit in the simulator; the next state is determined by the request stream and the cache evolution following the chosen action.

6.2 7-Dimensional State Vector

At each request routed to node j , the agent for node j observes:

Dim	Feature	Formula	Range
0	Base popularity	p_i	$[0, 1]$
1	Normalised size	$s_i/2500$	$[0.02, 1]$
2	Local cache indicator	$\mathbf{1}[c_i \in \text{cache}(j)]$	$\{0, 1\}$
3	Node load	ℓ_j	$[0, 1.5]$
4	Normalised latency	$L_{j,i,r_u}/200$	$\approx [0, 1]$
5	Cache overlap ratio	$\text{overlap}(j, \mathcal{N}_c)$	$[0, 1]$
6	Deadline slack (normalised)	$\max(0, L_j - D_1)/200$	$[0, 1]$

Dimension 2 is the single most informative feature: if the agent’s node already holds the content, serving locally is almost always optimal unless the node is heavily loaded. Dimension 6 directly encodes the degree to which a local serve would violate the deadline, pushing the agent towards cooperative or default forwarding when the deadline is at risk.

6.3 3-Action Space

Action 0 – SERVE_LOCAL (default) No override; the simulator’s standard routing decision tree applies (edge $\rightarrow$ cooperative $\rightarrow$ cloud).

Action 1 – FORWARD_DCCC Skip local serve attempt and jump directly to cooperative lookup. Useful when the agent detects it cannot meet D_1 but a cooperative node can meet D_2 .

Action 2 – DEFAULT_POLICY Equivalent to Action 0 in the current implementation; reserved for future extension (e.g., triggering pre-fetch).

The action space is intentionally narrow. Cache insertion decisions are handled by the ICE-style eviction logic and are not part of the action space; this decoupling produces significantly more stable Q-value convergence than joint routing-and-caching action spaces.

6.4 Network Architecture

Each DQN agent uses a two-hidden-layer fully connected network:

$$\hat{Q}(s, a; \theta) = W_3 \cdot \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 s + b_1) + b_2) + b_3 \quad (11)$$

where all hidden layers have width $H = 64$. A separate target network $\hat{Q}^-(s, a; \theta^-)$ is maintained and synced to θ at the end of each episode (hard update). The optimiser is Adam with learning rate $\eta = 10^{-3}$ and the loss is mean-squared error between the current Q-value and the Bellman target:

$$\mathcal{L}(\theta) = \left(r + \gamma \max_{a'} \hat{Q}^-(s', a'; \theta^-) - \hat{Q}(s, a; \theta) \right)^2 \quad (12)$$

with discount factor $\gamma = 0.95$.

6.5 Experience Replay

Each agent maintains a circular replay buffer of capacity 10 000 transitions $(s, a, r, s', \text{done})$. Mini-batches of size $B = 64$ are sampled uniformly at random. Training is performed after every request once the buffer holds at least B transitions.

A simplification is made: because the simulator does not expose the true next-state s' at the time the action is taken (the next state depends on the outcome), the current state s is used as a proxy for s' . This is a common approximation in online edge-caching DRL; the agent learns relative action preferences from the immediate reward rather than multi-step value estimates.

6.6 Reward Function

$$r = \begin{cases} +r_e & \text{if hit_type = edge} \\ +r_c & \text{if hit_type = cooperative} \\ -r_{cl} & \text{if hit_type = cloud} \end{cases} - r_l \cdot L + \begin{cases} +r_d & \text{if deadline met} \\ -r_d & \text{if deadline missed} \end{cases} \quad (13)$$

Default values: $r_e = 5.0$, $r_c = 3.0$, $r_{cl} = 5.0$, $r_l = 0.03$, $r_d = 5.0$. The latency coefficient r_l is small relative to the hit/miss terms so the agent prioritises cache efficiency; r_d creates a binary deadline signal that dominates when the deadline is very tight.

6.7 Training Loop

Algorithm 2 summarises the episodic training procedure. Training runs for $E = 50$ episodes. Each episode shuffles the first 1000 requests from the world’s request stream, runs a simulation with the DRL hook active, and after the episode syncs target networks and decays ε .

After training, `make_eval_hook` sets $\varepsilon = 0$ for all agents (pure greedy) and wraps them in a hook suitable for evaluation runs.

7 Evaluation Harness

7.1 Multi-Seed Runner

`run_multi_seed` iterates over all (policy, seed) pairs in the Cartesian product $\Pi \times \mathcal{S}$ where Π is the policy list and $\mathcal{S} = \{42, 43, 44, 45, 46\}$ is the seed set. For each pair:

1. A fresh world is built via `build_world(seed, config)`.
2. For `DEADLINE_DCCC`, DRL agents are trained independently per seed (so each seed yields its own trained policy, not a shared checkpoint).
3. After training, the world is rebuilt to a clean state and the evaluation run executes with $R_{\text{eval}} = 6000$ requests.
4. For all other policies, a single simulation run is performed.

This design means the DRL policy has been independently trained on each seed’s world, giving an honest estimate of per-seed variance rather than measuring the variance of a single trained checkpoint.

7.2 Output Files

Three CSV files are written to the output directory (default `results/`):

per_seed.csv One row per (policy, seed) with all 11 metrics. The primary raw data file.
summary.csv One row per policy with mean and 95% confidence interval half-width for each metric.

significance.csv One row per metric comparing DEADLINE_DCCC to the strongest non-deadline baseline on that metric, including relative change (%), paired t-test p-value, and Wilcoxon signed-rank p-value.

drl_training_history.csv (when DRL is trained) One row per (seed, episode) with cumulative reward, average loss, and ε .

7.3 Statistical Tests

7.3.1 95% Confidence Intervals

For each metric and policy, the 95% CI half-width is computed as:

$$\text{CI}_{95} = 1.96 \cdot \frac{\hat{\sigma}}{\sqrt{n}} \quad (14)$$

where n is the number of seeds and $\hat{\sigma}$ is the sample standard deviation. This uses the normal approximation; for $n = 5$ seeds the t-multiplier would be $t_{4,0.025} = 2.776$, which is more conservative. The normal approximation is used to avoid a hard dependency on `scipy`.

7.3.2 Paired t-Test

Let x_k and y_k denote the metric value for DEADLINE_DCCC and the best baseline respectively at seed k . The paired differences $d_k = x_k - y_k$ are tested against $H_0 : \mu_d = 0$ using:

$$t = \frac{\bar{d}}{\hat{\sigma}_d/\sqrt{n}}, \quad p = 2 \cdot \Phi(-|t|) \quad (15)$$

where Φ is the standard normal CDF computed via the Abramowitz & Stegun approximation for `erf`.

7.3.3 Wilcoxon Signed-Rank Test

The non-parametric complement to the paired t-test. The signed-rank statistic W^+ is computed using average ranks for ties, and the normal approximation

$$z = \frac{W^+ - n(n+1)/4}{\sqrt{n(n+1)(2n+1)/24}} \quad (16)$$

is used to derive a two-sided p-value. This test is more robust to the non-Gaussian distributions of latency and eviction counts.

7.4 Baseline Selection

For each metric, the best non-deadline baseline is selected as:

- The baseline policy with the highest mean for metrics where higher is better (hit rate, deadline satisfaction).

- The baseline policy with the lowest mean for metrics where lower is better (latency, cloud fetch rate, energy, evictions, load imbalance).

Policies in the `DEADLINE_*` family are excluded from baseline selection so that the significance test is always against a non-proposed competitor.

8 Configuration Reference

All 59 parameters are stored in `configs/simulation_config.json`. Every parameter can be overridden at runtime by passing a modified config path via `--config`. The tables below group parameters by functional area.

8.1 Network and Topology

Key	Type	Default	Description
<code>num_edge_nodes</code>	int	5	Number of edge nodes in the cluster
<code>num_regions</code>	int	3	Number of geographic regions
<code>num_contents</code>	int	120	Catalogue size
<code>cache_capacity_mb</code>	int	9000	Default per-node capacity (MB)
<code>capacity_profile</code>	list[int]	[9000,9000,2700,9000,2700]	Per-node capacity (overrides default)

8.2 Traffic Model

Key	Type	Default	Description
num_requests	int	6000	Total requests per simulation run
region_request_weights	list[float]	[0.60,0.25,0.15]	Region popularity weights (sum = 1)
region_weight_multiplier	float	2.5	Region-bias amplification for content selection
cross_region_prob	float	0.35	Prob. a node outside user region is a candidate
zipf_exponent	float	0.85	Zipf exponent for content popularity
content_size_min_mb	int	50	Minimum content size (MB)
content_size_max_mb	int	2500	Maximum content size (MB)
region_bias_min	float	0.1	Min value for per-region bias draw
region_bias_max	float	1.0	Max value for per-region bias draw
content_types	list[str]	[video,image,iot,audio]	Content type vocabulary

8.3 Latency and Deadline

Key	Type	Default	Description
strict_deadline_ms	int	35	Tier-1 (edge) deadline in ms
cooperative_deadline_ms	int	95	Tier-2 (cooperative) deadline in ms
base_edge_latency_ms	int	8	Base processing latency at edge
coop_path_latency_ms	int	35	Inter-node messaging overhead (ms)
cloud_latency_ms	int	120	Cloud base latency (ms)
transfer_ms_per_mb	float	0.012	Edge transfer coefficient (ms/MB)
cloud_transfer_ms_per_mb	float	0.025	Cloud transfer coefficient (ms/MB)
load_delay_ms	float	10.0	Per-unit load delay multiplier (ms)
region_delay_ms_per_hop	float	4.0	Inter-region delay per hop (ms)
max_node_load	float	1.5	Load ceiling for a node
load_increment_per_serve	float	0.25	Load increment per served request
load_decay	float	0.90	Per-step load decay multiplier
load_noise_amplitude	float	0.005	Stochastic jitter on load decay

8.4 Cache Management

Key	Type	Default	Description
secondary_cache_threshold	float	0.25	Min base_pop for secondary seed placement
ice_cache_gate	float	0.6	Min pop $\cdot \log(1 + s)$ for edge-hit re-insert
cooling_lambda	float	0.0004	Newton-cooling decay rate λ
popularity_request_scale	float	20.0	Demand normaliser σ in Newton popularity

8.5 Deadline-Aware Score Weights

Key	Type	Default	Description
score_w_popularity	float	2.0	Weight w_p for popularity term
score_w_hit_chance	float	6.0	Weight w_h for cache hit indicator
score_w_overlap	float	0.5	Weight w_o for cache overlap penalty
score_w_latency	float	0.08	Weight w_l for latency term
score_w_load_sq	float	1.2	Weight w_{ℓ^2} for quadratic load penalty
score_w_deadline_risk	float	0.4	Weight w_r for deadline-risk penalty

8.6 Energy Model

Key	Type	Default	Description
energy_local_factor	float	0.02	Energy coefficient for edge/coop serve (per MB)
energy_cloud_factor	float	0.09	Energy coefficient for cloud fetch (per MB)

8.7 DRL Hyperparameters

Key	Type	Default	Description
drl_state_size	int	7	Observation vector dimension
drl_action_size	int	3	Number of discrete actions
drl_learning_rate	float	0.001	Adam learning rate η
drl_gamma	float	0.95	Discount factor γ
drl_epsilon	float	1.0	Initial ε -greedy exploration rate
drl_epsilon_min	float	0.05	Minimum ε after decay
drl_epsilon_decay	float	0.97	Per-episode ε decay multiplier
drl_batch_size	int	64	Mini-batch size for Q-network update
drl_hidden_size	int	64	Hidden layer width in DQNNetwork
num_episodes	int	50	Number of training episodes
train_requests_per_episode	int	1000	Requests used per training episode
eval_requests	int	6000	Requests used in evaluation run

8.8 Reward Shaping

Key	Type	Default	Description
reward_edge_hit	float	5.0	Reward for serving from edge cache
reward_coop_hit	float	3.0	Reward for serving from cooperative node
reward_cloud_penalty	float	5.0	Penalty for cloud fetch
reward_latency_coef	float	0.03	Per-ms latency penalty coefficient
reward_deadline_bonus	float	5.0	Bonus/penalty for meeting/missing deadline

8.9 Experiment Harness

Key	Type	Default	Description
seeds	list[int]	[42,43,44,45,46]	Random seeds for multi-seed evaluation
policies	list[str]	(all 7 policies)	Policies included in experiment matrix

9 Reproducibility and Usage

9.1 Installation

```
git clone <repository-url>
cd deadline_dccc_project
pip install -r requirements.txt
# or: pip install -e .
```

Requirements include: `torch >= 2.0`, `pandas`, `matplotlib` (for plots/). The core simulator and heuristic policies have no dependencies beyond the Python standard library.

9.2 CLI Reference

The top-level entry point is `main.py`, which wraps `experiments/runner.py`. All flags are optional; omitting them uses the defaults from `configs/simulation_config.json`.

```
python main.py [--config PATH]
                [--seeds N [N ...]]
                [--policies NAME [NAME ...]]
                [--no-drl]
                [--episodes K]
                [--output-dir DIR]
                [--skip-plots]
```

9.3 Example Runs

9.3.1 Full Research Run

Runs all 7 policies over 5 seeds with DRL training enabled. This is the canonical experiment that produces all result files and plots.

```
python main.py
```

Expected output:

```
[run] policy=LRU seed=42
[run] policy=LRU seed=43
...
[run] policy=DEADLINE_DCCC seed=42
[DRL training] seed=42, episodes=50
Episode 1/ 50 | reward= -3241.75 | avg_loss= 0.4832 | eps=0.970
Episode 2/ 50 | reward= -1802.43 | avg_loss= 0.3106 | eps=0.941
...
Episode 50/ 50 | reward= 4891.22 | avg_loss= 0.0187 | eps=0.050
...
=== Summary (mean across seeds) ===
...
=== Significance (DEADLINE_DCCC vs best baseline) ===
...
Wrote plots to plots/*.png
```

9.3.2 Quick Smoke Test (No DRL)

Runs only LRU, LFU, and DCCC_HASH on a single seed without DRL training. Completes in seconds.

```
python main.py --no-drl --seeds 42 --policies LRU LFU DCCC_HASH
```

9.3.3 Deadline-Aware Ablation

Runs the three deadline-aware policies on seeds 42 and 43 with a shorter training run of 20 episodes.

```
python main.py --policies DEADLINE_HEURISTIC DEADLINE_NO_OVERLAP
DEADLINE_DCCC \
--seeds 42 43 --episodes 20
```

9.3.4 Single-Policy Evaluation with Custom Config

```
python main.py --config configs/simulation_config.json \
--policies DEADLINE_DCCC --seeds 42 43 44 45 46
```

9.3.5 Running the Experiment Harness Directly

```
python -m experiments.runner --no-drl --output-dir results_baseline
```

9.4 Output Directory Structure

After a full run the output directory contains:

```
results/
  per_seed.csv          # (policy, seed) x 11 metrics
  summary.csv           # per-policy mean +/- CI95
  significance.csv      # DEADLINE_DCCC vs best baseline
```

```

drl_training_history.csv # per (seed, episode) reward/loss/eps

plots/
performance_rates.png # hit/cloud/deadline rates bar chart
latency_breakdown.png # latency CDF or bar chart
# additional plots if plots/plots.py generates them

```

9.5 Extending the Framework

9.5.1 Adding a New Policy

1. Open `sim/simulator.py`.
2. Add a constant string for the policy name, e.g., `MY_POLICY = "MY_POLICY"`.
3. Add a branch in `_select_node` implementing the routing logic.
4. Add a branch in `_insert_after_serve` specifying the eviction algorithm (or reuse `add_lru/add_lfu/add_ice_style`).
5. Register the policy name in `configs/simulation_config.json` under "policies".

9.5.2 Custom DRL Hook

Any callable matching the signature

```

1 def my_hook(node, content, latency, deadline_ms,
2             candidate_nodes, all_nodes) -> int:
3     ... # return 0, 1, or 2

```

can be passed as `drl_hook` to `run_simulation`. This enables integration of alternative RL frameworks (e.g., `stable-baselines3`) without modifying the simulator core.

9.5.3 Programmatic API

```

1 from sim.simulator import build_world, run_simulation
2 from experiments.runner import load_config
3
4 config = load_config("configs/simulation_config.json")
5 world = build_world(seed=42, config=config)
6
7 # Evaluate LRU
8 metrics = run_simulation("LRU", seed=42, config=config, world=world)
9 print(metrics.to_row())
10
11 # Evaluate another policy on the exact same world
12 metrics2 = run_simulation("LFU", seed=42, config=config, world=world)

```

Note that `run_simulation` modifies the cache state of the nodes in `world` as a side effect. If multiple policies need to be compared on the same world without cross-contamination, build a fresh world for each policy (which is what `run_multi_seed` does). The current design allows world reuse only when the researcher explicitly wants stateful carryover between policy comparisons.

9.6 Reproducibility Checklist

- All random draws inside `build_world` use a single `random.Random(seed)` instance; no global random state is mutated.
- The simulation run-time RNG is seeded as `seed × 1 000 003 + 17` to be independent of the world-building RNG.
- PyTorch operations (DQN forward and backward pass) are *not* seeded by the simulator; for bit-exact DRL reproducibility, set `torch.manual_seed(seed)` before calling `train_drl_agents`.
- The load-noise jitter (5×10^{-3} amplitude) is seeded by the run-time RNG and is therefore fully reproducible given the seed.

10 Conclusion

EdgeSim-DCCC provides a complete, reproducible, and extensible Python simulation environment for deadline-aware cooperative edge caching research. Its key contributions as a simulator framework are:

1. **Paired world isolation:** every policy for a given seed processes the same request stream from the same initial cache state, making cross-policy comparisons statistically clean.
2. **Newton-cooling popularity:** a dynamic popularity model that combines Zipf-distributed base popularity with a request-demand signal and time decay, producing realistic cache-eviction pressure over the course of a simulation run.
3. **Two-tier deadline routing:** strict ($D_1 = 35$ ms) and cooperative ($D_2 = 95$ ms) deadline tiers mirror real MEC deployment constraints and create meaningful routing trade-offs that challenge both heuristic and learned policies.
4. **Decoupled DRL integration:** the `drl_hook` callback design keeps the simulator usable without PyTorch for ablation and baseline studies, while supporting full DQN training through the same loop with zero simulator changes.
5. **Statistical evaluation out-of-the-box:** the harness automatically runs 5-seed paired t-tests and Wilcoxon signed-rank tests, reducing researcher overhead and preventing p -hacking through a fixed test protocol.

Future extensions planned for the framework include: a federated learning mode where agents share gradients across nodes, a streaming request model where popularity drifts mid-run, network topology graphs replacing the flat region model, and a REST API for real-time dashboard monitoring of simulation runs.

The simulator is available as an open-source project. All results presented in associated papers can be reproduced by running `python main.py` with the default configuration.

A Module Index

Module path	Public API
sim/simulator.py	build_world, run_simulation, edge_latency_ms
models/content.py	Content dataclass
models/edge_node.py	EdgeNode: add_lru, add_lfu, add_ice_style
models/request.py	Request dataclass
policies/deadline_dccc_policy.py	newton_popularity, common_overlap, deadline_aware_score
drl/dqn_agent.py	DQNAgent, DQNNetwork
drl/trainer.py	train_drl_agents, make_eval_hook, build_drl_state
drl/replay_buffer.py	ReplayBuffer
experiments/runner.py	run_multi_seed, load_config
experiments/stats.py	mean_ci95, paired_t_pvalue, wilcoxon_signed_rank_pvalue
main.py	main() CLI entry point

B Policy Summary Table

Policy	Node selection	Coop. lookup	Eviction	DRL
LRU	Nearest latency	No	LRU	No
LFU	Nearest latency	No	LFU	No
DCCC_HASH	Hash ($c \bmod N_n$)	Yes ($\leq D_2$)	ICE	No
DCCC_NEAREST	Min latency	Yes ($\leq D_2$)	ICE	No
DEADLINE_HEURISTIC	Max $S(j)$	Yes ($\leq D_2$)	ICE	No
DEADLINE_NO_OVERLAP	Max $S(j)$, $w_o = 0$	Yes ($\leq D_2$)	ICE	No
DEADLINE_DCCC	Max $S(j)$ + DRL override	Yes ($\leq D_2$)	ICE	Yes

Algorithm 1 Main Simulation Loop (`run_simulation`)

Input: policy π , world $\mathcal{W}$, config $\mathcal{C}$, optional `drl_hook`

Output: SimMetrics

```
1: Initialise metrics counters; request_counts[c]  $\leftarrow$  0 for all  $c$ 
2: for ( $t, \text{req}$ ) in enumerate( $\mathcal{W}.\text{requests}$ ) do
3:    $c \leftarrow \mathcal{W}.\text{content\_map}[\text{req}.\text{content\_id}]$ 
4:   request_counts[c] += 1
5:   pop  $\leftarrow$  NEWTONPOPULARITY( $c, t, \text{request\_counts}[c]$ )
6:   for all node  $j \in \mathcal{W}.\text{nodes}$  do
7:      $\ell_j \leftarrow \max(0, \ell_j \cdot \lambda_{\text{decay}} + \text{noise})$   $\triangleright$  load decay + jitter
8:   end for
9:    $\mathcal{N}_c \leftarrow \text{nodes in req.region} \cup \{\text{nodes with } P(\text{cross}) < p_{\text{cross}}\}$ 
10:  ( $\hat{n}, \_$ )  $\leftarrow$  SELECTNODE( $\pi, \mathcal{N}_c, \mathcal{W}.\text{nodes}, c, \text{req.region}, \text{pop}, D_1$ )
11:   $L_{\hat{n}} \leftarrow \text{EDGE LATENCY}(\hat{n}, c, \text{req.region})$ 
12:  if  $\pi = \text{DEADLINE\_DCCC}$  and drl_hook  $\neq$  None then
13:     $a \leftarrow \text{drl\_hook}(\hat{n}, c, L_{\hat{n}}, \text{req.deadline}, \mathcal{N}_c, \mathcal{W}.\text{nodes})$ 
14:  else
15:     $a \leftarrow \text{None}$ 
16:  end if
17:  if  $a = 1$  (FORWARD\_DCCC) then
18:    attempt cooperative lookup; fall back to cloud
19:  else if  $\hat{n}.\text{has}(c)$  and  $L_{\hat{n}} \leq D_1$  then
20:    hit_type  $\leftarrow$  edge; latency  $\leftarrow L_{\hat{n}}$ 
21:  else if  $\pi \in \text{DCCC-family}$  and cooperative holder found within  $D_2$  then
22:    hit_type  $\leftarrow$  cooperative; latency  $\leftarrow L_{\text{coop}}$ 
23:  else
24:    hit_type  $\leftarrow$  cloud; latency  $\leftarrow L_{\text{cloud}}$ 
25:  end if
26:  evictions  $\leftarrow$  INSERTAFTERSERVE( $\pi, \text{cache\_target}, c, \text{pop}, t, \text{hit\_type}$ )
27:  update load, serves_per_node, hit counters, latencies, energy, evictions
28:  if log_callback  $\neq$  None then
29:    log_callback( $\{\text{request}, \text{policy}, \text{hit\_type}, \text{latency}, \dots\}$ )
30:  end if
31: end for
32: return metrics
```

Algorithm 2 DQN Training Loop (`train_drl_agents`)

Input: seed, config, world $\mathcal{W}$

Output: trained agents $\{\mathcal{A}_j\}_{j=0}^{N_n-1}$, history

- 1: Initialise one DQNAgent per node; $\varepsilon \leftarrow 1.0$
 - 2: **for** episode $e = 1, \dots, E$ **do**
 - 3: Shuffle $\mathcal{W}.\text{requests}[0 : T_{\text{train}}]$ with seed $s \cdot 7919 + e$
 - 4: Run `run_simulation` with `drl_hook` and `log_callback` active ▷ Per-request:
 agent selects action; `log_callback` trains on transition
 - 5: Sync target networks for all agents
 - 6: $\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \varepsilon_{\text{decay}})$ ($\varepsilon_{\text{decay}} = 0.97$, $\varepsilon_{\min} = 0.05$)
 - 7: Record episode reward, mean loss, ε
 - 8: **end for**
 - 9: Restore original request stream to $\mathcal{W}$
 - 10: **return** agents, history
-